EE45: Intro to Foundations of ML

Mod 1 - Lec 2: Data Fitting Concepts Part Deux

Overview: [Module 1] Data Fitting Concepts

- This week
  - How to measure "fit" of a model to data?: norms, distances, inner products
  - Example: k-means clustering

References:

- [VMLS]: Chapters 1–3
- Topics: Vectors, Inner product, Linear functions, Regression model, Norm and distance

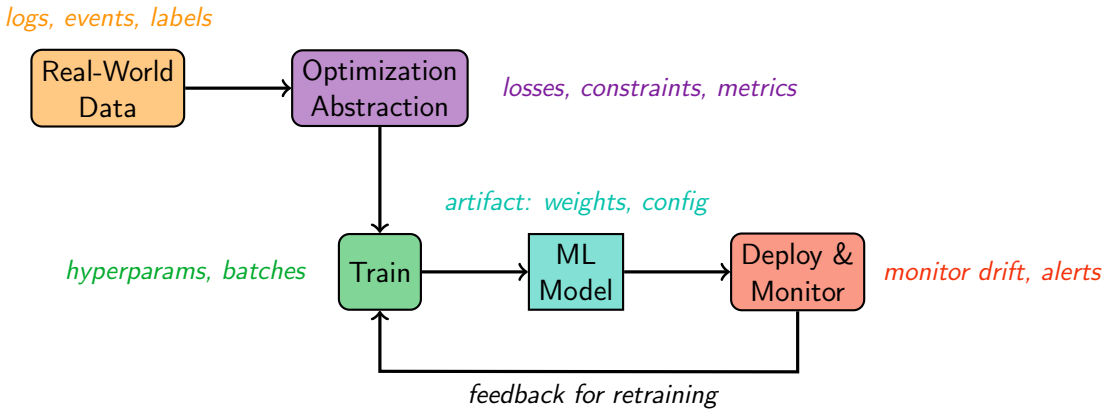
Content Summary

- Last time...
  - Formulate/abstract ML problems as an optimization problem
  - Similarity and distance measures as a key ingredient
  - Loss functions from distance measures such as Euclidean distance/norm, or absolute value, e.g.
  - Introduction to vector notation

Content Summary

- Last time...
  - Formulate/abstract ML problems as an optimization problem
  - Similarity and distance measures as a key ingredient
  - Loss functions from distance measures such as Euclidean distance/norm, or absolute value, e.g.
  - Introduction to vector notation
- This time...
  - reinforce the vector notation via examples
  -

ML workflow: Role of Linear Algebra & Optimization



Formulating a Learning Problem as Optimization

Model

A model is a $f_\theta : \mathcal{X} \to \mathcal{Y}$ recipe or rule that takes input data $x \in \mathcal{X}$ and produces a prediction $\hat{y} = f_\theta(x) \in \mathcal{Y}$.

- The quality of a model is measured by how well it fits the data. We can turn this into a number (an error or loss).
- We can then optimize that number.
- The loss is usually built from a distance (how far?) or a similarity (how aligned?).

Formulating a Learning Problem as Optimization

Model

A model is a $f_\theta : \mathcal{X} \to \mathcal{Y}$ recipe or rule that takes input data $x \in \mathcal{X}$ and produces a prediction $\hat{y} = f_\theta(x) \in \mathcal{Y}$.

- The quality of a model is measured by how well it fits the data. We can turn this into a number (an error or loss).
- We can then optimize that number.
- The loss is usually built from a distance (how far?) or a similarity (how aligned?).
- Machines need to measure similarity, and distances are the key
  - Norms and distances turn subjective ideas (are these two cats alike?) into math: $\|x - y\|$.
  - Different norms capture different ideas of similarity (Euclidean vs Manhattan vs cosine).

Vector Notation

- To measure similarity we first need to represent data as vectors.
- Last time we saw the example of digits from MNSIT represented as vectors based on their pixel values.
- Each data point with d features is just a point in $\mathbb{R}^d$.

Length of a vector

- We now have a way of writing data as vectors.
- Norm = measure of size/distance: Norms let us quantify how big a vector is or how far apart two vectors are.
- Examples:

Norm and distance

- A norm is a function that measures the length of a vector.
- Euclidean norm (2-norm) is $\|x\| = \sqrt{\sum_{i=1}^{n} x_i^2}$
- properties: $\alpha \in \mathbb{R}$ is a scalar, $x, y \in \mathbb{R}^n$ are vectors
  - homogeneity:
  - triangle inequality:
  - nonnegativity:
  - definiteness:
  - triangle inequality:

Inner Products

- inner product (or dot product) $\langle x, y \rangle : \mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}$ of n-vectors x and y is

$$\langle x, y \rangle = x^\top y = \sum_{i=1}^{n} x_i y_i$$

  satisfying:
  - **Linearity:**
  - **Symmetry:**
  - **Positivity:**
  - **Cauchy-Schwarz inequality:**
  - **Homogeneity:**

- Examples:

Examples

Matrix–Vector Multiplication as Inner Products

Inner Products Induce Norms

- Every inner product induces a norm: $\|x\| = \sqrt{\langle x, x \rangle}$
- In the "standard" inner product:

- To check: check that if we deine the norm as $\|x\| = \langle x, x \rangle^{1/2}$ then all the norm properties hold (homogeneity, triangle inequality, etc.)

- To check: $\|x\|_M = \sqrt{x^\top M x}$ for any symmetric positive definite matrix M is a norm.
- However, not every norm is induced by an inner product.

Why Norms?: Norms as Distances & Measures of Error

- Measuring error between prediction and truth:

$$\text{Error} = \|\hat{y} - y\|$$

- Example: residuals $r_i = y_i - \hat{y}_i$ and error $\|r\|_2 = \sqrt{\sum_{i=1}^{n} r_i^2}$
- Different norms $\Rightarrow$ different geometry of error

Norms

Define residual vector $r = (r_1, \ldots, r_n)^\top \in \mathbb{R}^n$.

Choosing Norms

- $\|\cdot\|_2$: "mean fit" $\rightarrow$ sensitive to outliers, emphasizes large errors (squares them)
- $\|\cdot\|_1$: "median fit" $\rightarrow$ more robust.
- $\|\cdot\|_\infty$: "worst-case fit" $\rightarrow$ minimizes maximum deviation.

Error Growth

- ℓ_2 (squared error): Errors grow quadratically.

$$\|r\|_2^2 = \sum_i r_i^2$$

  - Residual = 10 $\rightarrow$ contributes 100; Residual = 100 $\rightarrow$ contributes 10,000
- ℓ_1 (absolute error): Errors grow linearly.

$$\|r\|_1 = \sum_i |r_i|$$

  - Residual = 10 $\rightarrow$ contributes 10; Residual = 100 $\rightarrow$ contributes 100
- ℓ_∞: Errors grow like the largest residual.

$$\|r\|_\infty = \max_i |r_i|$$

Norms as Loss Functions

- $\ell_2(\hat{y}, y) = \|\hat{y} - y\|_2^2$

Norms as Loss Functions

- $\ell_2(\hat{y}, y) = \|\hat{y} - y\|_2^2$
  - Use case: Housing price prediction, where most houses follow a trend (e.g., price per square foot) but a few are extreme (mansions, distressed sales).
  - ℓ_1 focuses on the median error, while ℓ_2 emphasizes the mean error.

Norms as Loss Functions

- $\ell_2(\hat{y}, y) = \|\hat{y} - y\|_2^2$
  - Use case: Housing price prediction, where most houses follow a trend (e.g., price per square foot) but a few are extreme (mansions, distressed sales).
  - ℓ_1 focuses on the median error, while ℓ_2 emphasizes the mean error.
  - Use case: Image reconstruction, autoencoders, super-resolution.
  - ℓ_2 Produces smooth, averaged outputs (ℓ_1 give meadian like images, blocky)

Norms as Loss Functions

- $\ell_2(\hat{y}, y) = \|\hat{y} - y\|_2^2$
  - Use case: Housing price prediction, where most houses follow a trend (e.g., price per square foot) but a few are extreme (mansions, distressed sales).
  - ℓ_1 focuses on the median error, while ℓ_2 emphasizes the mean error.
  - Use case: Image reconstruction, autoencoders, super-resolution.
  - ℓ_2 Produces smooth, averaged outputs (ℓ_1 give meadian like images, blocky)
  - Use case: Standard regression in controlled experiments (e.g., predicting exam scores from study hours).
  - ℓ_2 Data has small, symmetric errors; OLS is unbiased and optimal under Gauss–Markov assumptions.

Norms as Loss Functions

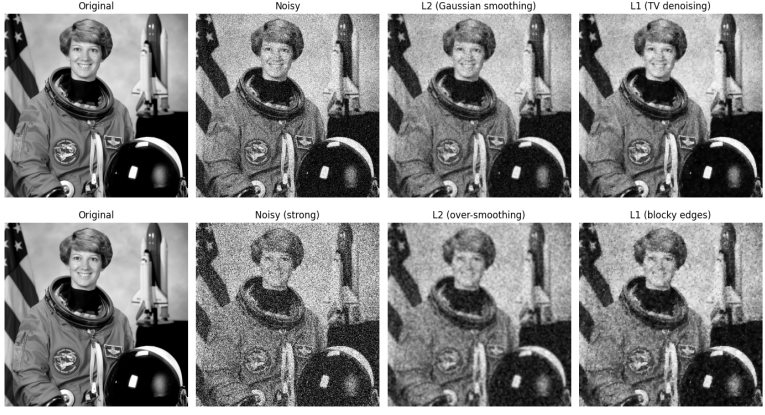
- $\ell_2(\hat{y}, y) = \|\hat{y} - y\|_2^2$
  - Use case: Housing price prediction, where most houses follow a trend (e.g., price per square foot) but a few are extreme (mansions, distressed sales).
  - ℓ_1 focuses on the median error, while ℓ_2 emphasizes the mean error.
  - Use case: Image reconstruction, autoencoders, super-resolution.
  - ℓ_2 Produces smooth, averaged outputs (ℓ_1 give meadian like images, blocky)
  - Use case: Standard regression in controlled experiments (e.g., predicting exam scores from study hours).
  - ℓ_2 Data has small, symmetric errors; OLS is unbiased and optimal under Gauss–Markov assumptions.
- $\ell_1(\hat{y}, y) = \|\hat{y} - y\|_1$
  - Use case: Housing price prediction, where most houses follow a trend (e.g., price per square foot) but a few are extreme (mansions, distressed sales).
  - ℓ_1 focuses on the median error, while ℓ_2 emphasizes the mean error.

Norms as Loss Functions

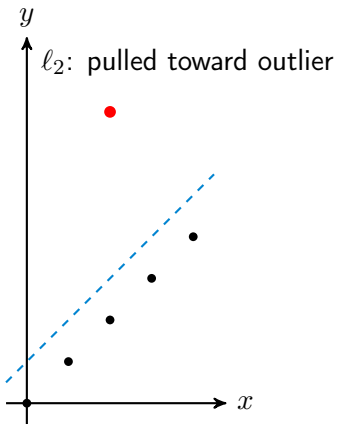
- $\ell_2(\hat{y}, y) = \|\hat{y} - y\|_2^2$
  - Use case: Housing price prediction, where most houses follow a trend (e.g., price per square foot) but a few are extreme (mansions, distressed sales).
  - ℓ_1 focuses on the median error, while ℓ_2 emphasizes the mean error.
  - Use case: Image reconstruction, autoencoders, super-resolution.
  - ℓ_2 Produces smooth, averaged outputs (ℓ_1 give meadian like images, blocky)
  - Use case: Standard regression in controlled experiments (e.g., predicting exam scores from study hours).
  - ℓ_2 Data has small, symmetric errors; OLS is unbiased and optimal under Gauss–Markov assumptions.
- $\ell_1(\hat{y}, y) = \|\hat{y} - y\|_1$
  - Use case: Housing price prediction, where most houses follow a trend (e.g., price per square foot) but a few are extreme (mansions, distressed sales).
  - ℓ_1 focuses on the median error, while ℓ_2 emphasizes the mean error.
  - Use case: Image reconstruction in medical imaging (MRI, CT) where data is undersampled.
  - ℓ_1 Encourages sparsity and robustness.

Example Image Reconstruction

- ℓ_2 produces smooth, averaged outputs.
- ℓ_1 produces blocky, median-like images.

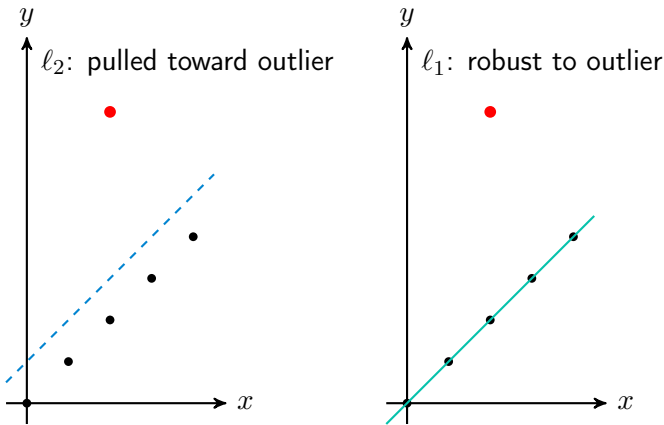


Fitting a line with one extreme outlier: ℓ_2 vs ℓ_1 vs ℓ_∞



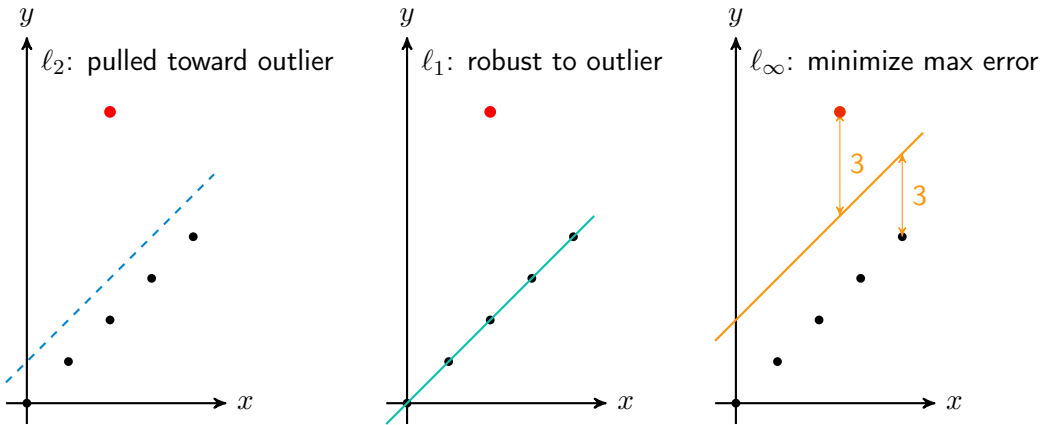
- ℓ_2: Large errors dominate (quadratic), so the fit bends toward the outlier.
- ℓ_1: Linear growth; outlier has limited pull, line stays with the bulk.
- ℓ_∞: *minimizes the largest absolute residual*

Fitting a line with one extreme outlier: ℓ_2 vs ℓ_1 vs ℓ_∞



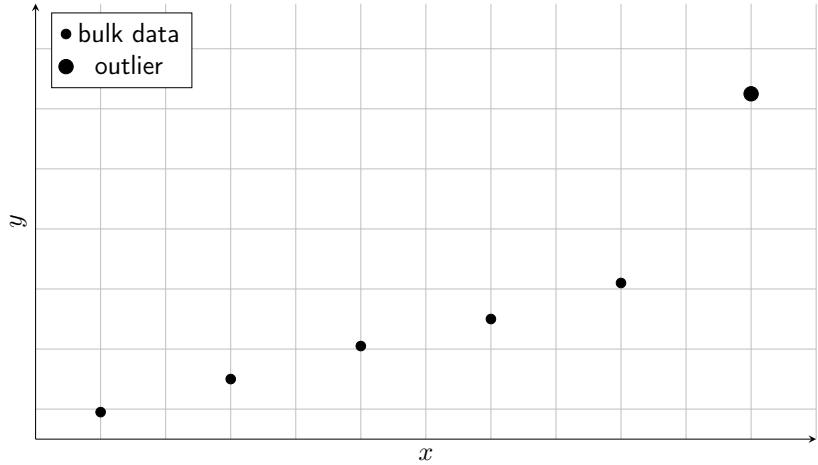
- ℓ_2: Large errors dominate (quadratic), so the fit bends toward the outlier.
- ℓ_1: Linear growth; outlier has limited pull, line stays with the bulk.
- ℓ_∞: *minimizes the largest absolute residual*

Fitting a line with one extreme outlier: ℓ_2 vs ℓ_1 vs ℓ_∞



- ℓ_2: Large errors dominate (quadratic), so the fit bends toward the outlier.
- ℓ_1: Linear growth; outlier has limited pull, line stays with the bulk.
- ℓ_∞: *minimizes the largest absolute residual*

Choosing Norm Impacts Model Fit: Let's apply the logic



Note: Without this outlier, ℓ_2 and ℓ_1 give nearly identical fits. The outlier is the stress test.

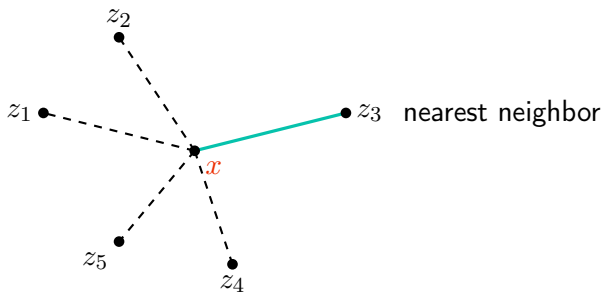
Why Distance Matters

- We saw norms as a way to define losses based on errors.
- Distances define what it means for points to be "close" or "similar."
- Nearest Neighbor and k-Means clustering rely entirely on this notion.
- Different distances $\Rightarrow$ different neighbors and clusters.

Feature distance and nearest neighbors

- if x, y are feature vectors for two entities, $\|x - y\|$ is the *feature distance*
- for vectors $z_1, \ldots, z_m$, z_j is nearest neighbor of x if

$$\|x - z_j\| \leq \|x - z_i\|, \quad i = 1, \ldots, m$$



Example: document dissimilarity

- 5 Wikipedia articles: 'Veterans Day', 'Memorial Day', 'Academy Awards', 'Golden Globe Awards', 'Super Bowl'
- word count histograms, dictionary of 4423 words (more setup details in sec 4.4.5 of VMLS)
- pairwise distances shown below:

	Veterans' day	Memorial day	Academy A.	Golden G.	Super Bowl
Veterans' day	0	0.095	0.130	0.153	0.170
Memorial day	0.095	0	0.122	0.147	0.164
Academy A.	0.130	0.122	0	0.108	0.164
Golden G.	0.153	0.147	0.108	0	0.181
Super Bowl	0.170	0.164	0.164	0.181	0

- we'll revisit this example later

Document Dissimilarity

- We may want to cluster documents into groups.
- e..g, awards, sports, holidays, etc.
- We can use k-means to cluster documents into k groups.

K-Means (Euclidean)

Goal. Partition data into k clusters by minimizing

$$J(\{\mu_j\}, z) \;=\; \sum_{i=1}^{n} \left\| x_i - \mu_{z_i} \right\|_2^2$$

Effect of Distance on K-Means

Euclidean (ℓ_2):

- Clusters are round (balls).
- Center = arithmetic mean.
- Sensitive to scale of features.

Manhattan (ℓ_1):

- Clusters look diamond-shaped.
- Center = componentwise median.
- More robust to outliers.

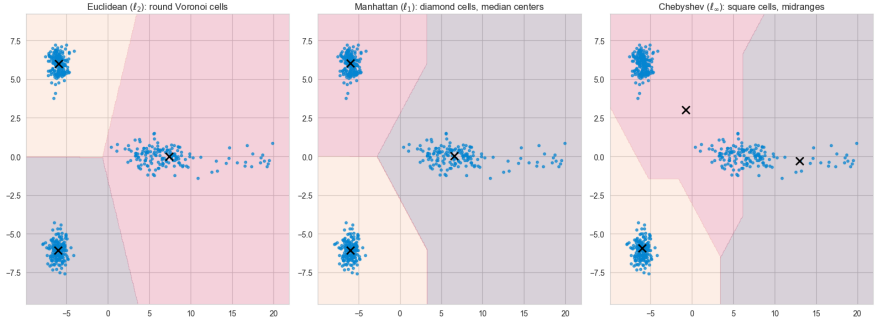
Cosine similarity:

- Clusters by angle, not magnitude.
- Center = normalized mean direction.
- Common for text or embeddings.

General lesson:

- Choice of distance = inductive bias.
- Different geometries $\Rightarrow$ different clusters.

K-Means with Different Distances



- ℓ_2: round clusters, center on the elongated cluster is pulled toward the outliers (mean).
- ℓ_1: diamond clusters, center sits closer to the componentwise median, resisting the long tail.
- ℓ_∞: angular clusters, center sits at the midrange (avg of min/max per coordinate), so it shifts differently again.

k-NN vs k-Means

k-Nearest Neighbors (k-NN)

- **Supervised learning algorithm**
- Used for classification (and regression)
- Predicts label of a new point by majority vote of its k nearest neighbors
- Relies on a distance metric (Euclidean, Manhattan, etc.)
- No training phase beyond storing the data

k-Means Clustering

- **Unsupervised learning algorithm**
- Used for clustering unlabeled data
- Iteratively assigns points to k clusters and updates cluster centers
- Objective: minimize within-cluster variance
- Training = alternating assignment and update until convergence

Both use k and a distance metric, but serve very different purposes.

Document Dissimilarity: k-NN vs k-Means

Example corpus:

Veterans Day, Memorial Day, Academy Awards, Golden Globes, Super Bowl

k-Means (unsupervised)

- Input: pairwise document distances (from word-count histograms).
- Goal: cluster into $k = 3$ groups.
- Outcome:
  - Veterans Day, Memorial Day $\rightarrow$ Holidays
  - Academy Awards, Golden Globes $\rightarrow$ Awards
  - Super Bowl $\rightarrow$ Sports
- No labels required.

k-NN (supervised)

- Assume we already have labels: Holiday, Award Show, Sports.
- New document: "Labor Day."
- Compute distances to all 5 documents.
- Nearest neighbors (Veterans Day, Memorial Day).
- Majority label = Holiday $\Rightarrow$ classify as Holiday.

Same distance representation, two different uses.

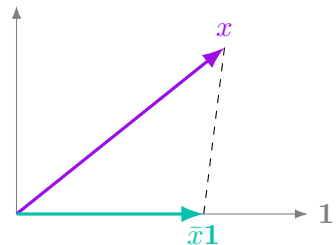
(If time permits) Standardizing Data

Example:Mean as Inner Product

- The mean is the “balance point” (center of mass).
- The sample mean is

$$\mu = \bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i = \frac{\langle x, \mathbf{1} \rangle}{\langle \mathbf{1}, \mathbf{1} \rangle}$$

- Geometrically: $\bar{x} \cdot \mathbf{1}$ is the projection of x onto the "span" of $\mathbf{1}$
- Intuition: the mean is the "best constant approximation" of the data



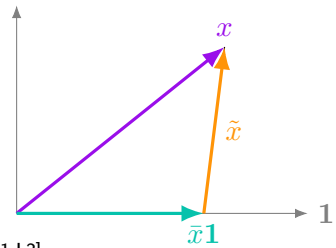
Centering Feature Vectors

Given $x \in \mathbb{R}^n$ and $\mathbf{1} = (1, 1, \ldots, 1)^\top \in \mathbb{R}^n$:

$$\tilde{x} = x - \bar{x}\,\mathbf{1} = x - \frac{1}{n}\,\mathbf{1}^\top x\,\mathbf{1} = x - \frac{1}{n}\,\mathbf{1}\mathbf{1}^\top x = \left(I - \tfrac{1}{n}\mathbf{1}\mathbf{1}^\top\right)x$$

- $\mathbf{1}\mathbf{1}^\top \in \mathbb{R}^{n\times n}$ is the all-ones matrix.
- $H = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$ is the centering matrix.

- H is symmetric and idempotent ($H^2 = H$): an orthogonal projection.
- Geometrically: projects x onto the subspace orthogonal to $\mathbf{1}$.



Example:Variance and Standard Deviation

- The centered data vector is $x - \mu\mathbf{1}$
- The standard deviation is the "typical distance" of points from that balance point.
  - Small SD $\rightarrow$ data clustered tightly around the mean.
  - Large SD $\rightarrow$ data spread out.
- If you pick a random data point, the standard deviation is roughly the size of the "typical error" you'd make if you always guessed the mean.
- Think of the mean as a reference position. The standard deviation is like the moment of inertia—how far mass is spread out from the center. Same "center," but very different spread.
- The variance is the scaled average squared deviation

Example:Variance and Standard Deviation

- The centered data vector is $\tilde{x} = x - \bar{x}\mathbf{1} = Hx$, where $H = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$ is the centering matrix.
- Its squared norm gives the total squared deviation:
- The variance is the scaled average squared deviation:
- Standard deviation (= avg length of the projected (centered) vector):

Standardizing Data

Given data $x = (x_1, \ldots, x_n)$ with mean $\bar{x}$ and std. deviation s, the standardized data (z-scores) are

Properties

- Centered: mean$(z) = 0$
- Scaled: sd$(z) = 1$
- Interpretable in standard deviation units.
- Comparable across different variables

Standardizing Data (z-scores)

Given $x \in \mathbb{R}^n$, mean $\bar{x}$, and centering matrix the centered data is

The standard deviation is

Standardized data (z-scores):

Why (and When Not) to Standardize Data

- **Why standardize:**
  - **Comparability across features** Different units (kg, $, counts) $\rightarrow$ put in common std dev units.
  - **Geometry matters** kNN, SVM, PCA use distances/dot products; large-scale features dominate otherwise.
  - **Optimization stability** Gradient descent converges faster; loss surface becomes "rounder."
  - **Interpretability** Coefficients are comparable across features.

Why (and When Not) to Standardize Data

- **Why standardize:**
  - **Comparability across features** Different units (kg, $, counts) $\rightarrow$ put in common std dev units.
  - **Geometry matters** kNN, SVM, PCA use distances/dot products; large-scale features dominate otherwise.
  - **Optimization stability** Gradient descent converges faster; loss surface becomes "rounder."
  - **Interpretability** Coefficients are comparable across features.
- **When not to standardize:**
  - **Categorical features** encoded as one-hot or integers $\rightarrow$ standardizing breaks their meaning.
  - **Features with physical meaning in the model** e.g. age in years, probability values, counts with direct interpretability.
  - **Tree-based models** (decision trees, random forests, gradient boosting) $\rightarrow$ splits are order-based, not distance-based; scaling irrelevant.